

Difference between Thread & Process

A **process** is a running program with its own memory/address space.

A **thread** is an execution unit inside a process. A process can have one or multiple threads.

Threads in the same process **share the process's memory**, so they can access the same global variables and heap data.

But each thread has its **own stack, registers, and program counter**.

Example

```
Codexion process
|
├── Coder thread 1
├── Coder thread 2
├── Coder thread 3
└── Coder thread 4
```

All coder threads can access:

```
dongles[0]
dongles[1]
shared_data
```

because they belong to the same process.

But each thread has its own stack:

```
Thread 1 → Stack 1
Thread 2 → Stack 2
Thread 3 → Stack 3
```

Easy rule

```
Process → separate memory
Threads → shared memory, separate stacks
```

Evaluation answer

A process is a running program with its own address space. A thread is an execution unit inside a process. Threads of the same process share its memory, while each thread has its own stack, registers, and program counter.

Thread Types: User-Level & Kernel-Level

There are two ways threads can be managed.

User-level threads are managed by a library in user space. The kernel doesn't directly manage each user thread.

Kernel-level threads are known and managed by the operating system kernel. The kernel scheduler can schedule them independently.

Example

Think of a manager.

User-level:

```
graph TD; A[Your program] --> B[Thread library]; B --> C[Kernel];
```

The diagram illustrates user-level thread management. It shows a vertical flow starting with 'Your program', followed by a downward arrow to 'Thread library', and another downward arrow to 'Kernel'. This indicates that the thread library acts as an intermediary between the user program and the kernel.

The library decides how its threads are managed.

Kernel-level:

```
graph TD; A[Your program] --> B[Kernel]; B --- C[Thread A]; B --- D[Thread B]; B --- E[Thread C];
```

The diagram illustrates kernel-level thread management. It shows a vertical flow starting with 'Your program', followed by a downward arrow to 'Kernel'. From the 'Kernel' level, three horizontal lines branch out to the right, labeled 'Thread A', 'Thread B', and 'Thread C' respectively. This indicates that the kernel is directly aware of and manages individual threads.

The kernel knows about the threads and schedules them.

Important point

With kernel-level threads, if you have multiple CPU cores:

Core 1 → Thread A
Core 2 → Thread B

they can actually execute in parallel.

If Thread A blocks, Thread B can still continue.

Evaluation answer

User-level threads are managed by a user-space thread library, while kernel-level threads are managed and scheduled by the operating system kernel.

CPU Scheduler

The **CPU scheduler** is part of the operating system kernel.

Its job is to decide:

Which runnable thread should run on the CPU now?

Example

Suppose three threads are ready:

Thread A
Thread B
Thread C

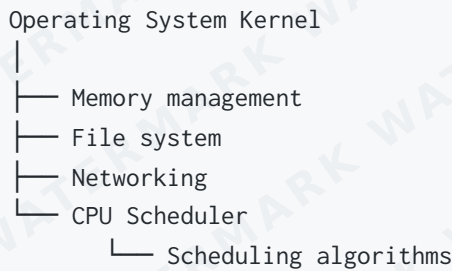
The scheduler decides:

CPU → A
CPU → B
CPU → C
CPU → A
CPU → B
CPU → A
CPU → C
...
...

On a multi-core CPU:

Core 1 → A
Core 2 → B

Where is it?

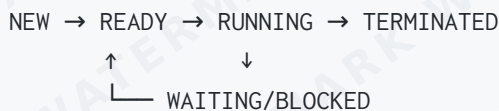


Evaluation answer

The CPU scheduler is a component of the operating system kernel that decides which runnable thread or process gets CPU time.

Thread Life Cycle

A thread doesn't just magically exist and start running. It goes through different states.



What each state means

NEW

The thread has been created.

READY

The thread is ready to run but is waiting for the CPU.

RUNNING

The thread is currently executing on a CPU.

WAITING/BLOCKED

The thread cannot continue because it is waiting for something.

For example, in Codexion:

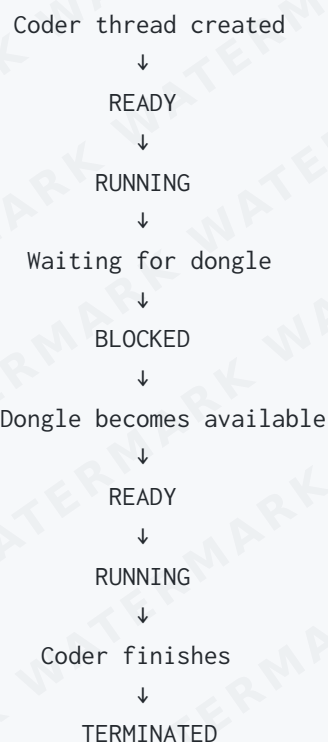
```
pthread_mutex_lock(&dongle->mutex);
```

If another coder owns that mutex, the thread may block waiting for it.

TERMINATED

The thread has finished executing.

Example



Evaluation answer

A thread can be created, become ready, run, become blocked while waiting for a resource or event, return to the ready state, and eventually terminate.

Atomic Instruction

Atomic means an operation happens **as one indivisible step**. Another thread cannot interfere in the middle of that operation.

Consider:

```
x++;
```

It may actually happen like this:

```
x = 5
```

Thread A:

```
reads x → 5
```

```
adds 1 → 6
```

```
writes x → 6
```

Thread B:

```
reads x → 6
```

```
adds 1 → 7
```

```
writes x → 7
```

That works because B reads the value **after A finishes**.

But without atomicity, this can happen:

```
x = 5
```

Thread A:

```
reads x → 5
```

```
adds 1 → 6
```

```
writes x → 6
```

Thread B:

```
reads x → 5
```

```
adds 1 → 6
```

```
writes x → 6
```

```
Final x = 6
```

We expected:

```
5 + 1 + 1 = 7
```

but got:

```
6
```

What atomic instruction does

An atomic increment makes the whole operation indivisible:

x = 5

Thread A:

[read + add + write] → 6

Thread B:

[read + add + write] → 7

Thread B **cannot enter the middle of A's operation.**

So:

Thread A: [x++]

Thread B: [x++]

rather than:

Thread A: read

Thread B: read

Thread A: add

Thread B: add

Thread A: write

Thread B: write

Evaluation answer

An atomic instruction is an operation that executes indivisibly. Other threads cannot interfere in the middle of it, which prevents problems such as lost updates when multiple threads modify the same variable.

Context Save

When a thread stops running because it sleeps, blocks, or gets switched out, the kernel needs to remember **where the thread was and how it was executing**.

The kernel saves its **CPU context**.

This includes things such as:

Program counter

Stack pointer

CPU registers

Other CPU state

The thread's actual stack remains in memory. The kernel doesn't copy the whole stack every time the thread sleeps.

Who saves it?

The kernel.

Where?

Conceptually, in a **Thread Control Block (TCB)** or equivalent kernel-managed thread structure.

PCB and TCB

PCB = Process Control Block

Contains process-related information.

```
PCB
├── Process state
├── Memory information
├── Open files
└── Thread information
```

TCB = Thread Control Block

Contains thread-related information.

```
TCB
├── Thread state
├── Program counter
├── Stack pointer
├── CPU registers
└── Scheduling information
```

Example

Suppose:

```
printf("Hello");
sleep(2);
printf("World");
```

When the thread sleeps, the kernel needs to remember the state needed to resume it so it can eventually continue with:

```
printf("World");
```

Evaluation answer

When a thread blocks or sleeps, the kernel saves its CPU execution context, such as its program counter, stack pointer, and registers, in kernel-managed data associated with the thread, commonly described as a TCB.

Context Switch

A **context switch** happens when the CPU stops executing one thread and starts executing another.

For example:

```
Thread A running
  ↓
Kernel saves A's context
  ↓
Kernel restores B's context
  ↓
Thread B running
```

Later:

```
Thread B running
  ↓
save B
  ↓
restore A
  ↓
Thread A continues
```

The **scheduler decides which thread should run**, and the kernel performs the context-switching work.

Example

With one CPU core:

```
Thread A → Thread B → Thread C → Thread A → ...
```

The CPU is switching between them.

Important

A context switch has a **cost** because saving and restoring CPU state takes time.

Evaluation answer

A context switch is when the kernel saves the context of the currently running thread and restores the context of another thread so the CPU can switch execution between them.

Concurrency vs Parallelism

Concurrency

Multiple threads are **making progress during the same period of time**, but they don't necessarily execute at the exact same instant.

With one core:

```
Core 1
↓
A → B → C → A → B → C → ...
```

The scheduler switches between them.

Parallelism

Multiple threads are **actually executing at the same time** on different CPU cores.

```
Core 1 → Thread A
Core 2 → Thread B
Core 3 → Thread C
```

Simple example

Imagine three coders.

One core:

```
Coder A → compile
Coder B → compile
```

Coder C → compile
Coder A → compile

They are concurrent.

Three cores:

Core 1 → Coder A → compile
Core 2 → Coder B → compile
Core 3 → Coder C → compile

They can execute in parallel.

Evaluation answer

Concurrency means multiple threads can make progress during the same period of time, possibly by switching between them. Parallelism means multiple threads are executing simultaneously on different CPU cores.

The whole picture

These concepts connect together like this:

